05 - Frontend Design

Generated 2026-10-14 from 05-FRONTEND-DESIGN.md - information only, not medical advice.

Contents

Principles
Screens
Behaviour details
Tests

Location: `frontend/` · **Stack:** plain HTML/CSS/JS, no build step, no runtime dependencies. `package.json` holds dev tooling only (ESLint, Playwright). Production: unprivileged nginx (uid 10001, read-only FS) serves `src/` and proxies `/api/` to the backend (same origin -> strict CSP, no CORS).

Principles

- **Safety visible:** the disclaimer bar is always on screen; an emergency banner is pinned on top when triage fires; refusal cards explain why and where to get help.
- **Honest output:** model text is only inserted with `textContent` / `createTextNode` (ESLint forbids `innerHTML`, `insertAdjacentHTML`, `document.write`, `eval`). Citations are chips linked to source cards.
- **No external assets:** CSP `default-src 'self'`, no inline script/style, `connect-src` adds only Supabase Auth.

Screens

Screen	Elements	Roles
Sign in	Split screen: promises + email/password form (Supabase password grant, no SDK)	all
Chat	History sidebar, streamed phases (safety -> searching -> generating -> verifying), answer card, "Sources and limitations", "How this answer was produced" (agent trace), evidence-report download, "Search in" filters	all
Research mode	Checkbox in filters; same answer card with web sources cited by URL	clinician, admin
Documents	Drag-and-drop upload with provenance form, documents table with delete	admin
Evaluation	Score rings (quality, safety gates, review verdicts), labelled synthetic offline evaluation	admin
Review	Queue of sampled (redacted) answers with verdict buttons and notes	clinician, admin

Screenshots: `docs/screenshots/` (regenerated by `npm run screenshots`).

Behaviour details

- Streaming: `fetch` + manual SSE parsing of `POST /api/chat/stream` (`event: phase|answer|error`).
- Session: token in `sessionStorage` (wrapped in try/catch), refresh via Supabase refresh grant.
- Boot message visible until the script runs; explains how to open the app correctly if it never does.

- Accessibility: skip link, landmarks, `aria-live` messages, focus outlines, keyboard drop zone, labelled controls, reduced-motion support; light/dark themes (system default + toggle); mobile drawer below the disclaimer.

- Agent trace: `renderTrace()` adds a collapsed `<details class="agent-trace">` under every answer that carries `safety.trace`. Summary: "How this answer was produced (N agent steps, T s)"; one row per step with a status dot, agent, outcome, duration in ms and detail; the status is also given as screen-reader text (done / attention / stopped here / skipped). Text nodes only. Status classes are prefixed `st-` (`st-ok`, `st-warn`, `st-stop`, `st-skip`) because a plain `.skip` class collided with the skip-link style and hid skipped rows. Screenshot: `docs/screenshots/15-agent-trace.png`.

- Cache: HTML/JS/CSS served with `Cache-Control: no-cache` (ETag revalidation) so stale JS never runs.

Tests

`frontend/tests/ui.spec.js` (10 Playwright tests in the installed Edge against the backend demo server): sign-in without script errors, cited answer, emergency first, dosing refusal, injection blocked, role gating, report download, mobile disclaimer never covered, agent trace lists every agent step (all rows visible), agent trace shows where an emergency stopped the pipeline. ESLint enforces the no-HTML-injection rule.